05

DOM: manipulando a página

Apostila do aluno · duração da aula: 2h

O que você vai aprender

- 1. O que é o DOM?
- 2. Selecionando elementos
- 3. Lendo e modificando
- 4. Navegando pela árvore
- 5. Eventos: reagindo ao usuário

Antes de começar

Esta aula junta tudo o que vimos: variáveis, condicionais, arrays, laços e funções (os **callbacks** do `forEach` voltam no `addEventListener`). No site **site-alunos-nu.vercel.app** cada exemplo e exercício tem uma **prévia da página**: o seu código roda dentro dela e dá para clicar e digitar.

Prof. Walter Caciano · Greenn School

1 O que é o DOM?

DOM (Document Object Model, ou Modelo de Objeto de Documento) é a forma como o navegador representa a página: ele lê o **HTML** e cria **objetos** que o JavaScript pode ler e alterar. É a ponte entre a página e o seu código.

```
<body>
  <h1 id="titulo">Green School</h1>
  <ul class="lista">
    <li>HTML</li>
    <li>CSS</li>
  </ul>
  <button id="btn">Clique</button>
</body>
```

Cada parte do HTML vira um **nó** de uma árvore: o `body` é **pai** do `h1`, do `ul` e do `button`; os `li` são **filhos** do `ul` e **irmãos** entre si. É assim que o JavaScript "caminha" pela página.

Termo	O que é
Elemento	a tag inteira, que vira um objeto que podemos manipular
Atributo	informação extra dentro da tag: `id`, `class`, `href`, `src`...
Nó de texto	o conteúdo escrito dentro do elemento

Todo acesso começa pelo objeto global `document`: `document.title` (título da aba), `document.body` e os métodos de seleção.

2 Selecionando elementos

```
document.getElementById("titulo");         // 1 elemento, pelo id (SEM #) ou null
document.getElementsByClassName("item");   // HTMLCollection (SEM ponto), sem forEach
document.getElementsByTagName("li");       // HTMLCollection (sem < >)

document.querySelector("#titulo");         // 1º que combina com o seletor CSS ou null
document.querySelectorAll(".item");        // NodeList com todos, TEM forEach
```

	querySelector()	querySelectorAll()
Retorna	1 elemento	NodeList (lista)
Sem resultado	`null`	lista vazia (length 0)
Quando usar	preciso de um só	preciso de vários

> **Se funciona no CSS, funciona no querySelector:** `"ul > li:first-child"`, `"input[type='text']"`, `".card .titulo"`. Para usar `forEach` numa HTMLCollection, converta com `Array.from(itens)`.

3 Lendo e modificando

Texto, HTML e valor: `textContent` lê/troca só o texto (seguro); `innerHTML` lê/troca o HTML de dentro (as tags são interpretadas: não use com texto digitado pelo usuário); `value` é o valor de um input, sempre uma **string**.

```
titulo.textContent = "Olá, DOM!";
lista.innerHTML = "<li>Novo item</li>";
console.log(nome.value);              // texto digitado
```

Classes com classList (prefira a `className`, que apaga as outras classes):

```
card.classList.add("ativo");          // adiciona
card.classList.remove("oculto");      // remove
card.classList.toggle("destaque");    // se tem, tira; se não tem, põe
card.classList.contains("ativo");     // true ou false
```

Estilos com style: nomes em camelCase (`backgroundColor`, não `background-color`) e valores sempre em texto com unidade.

```
titulo.style.color = "green";
titulo.style.backgroundColor = "#111";
titulo.style.fontSize = "24px";
```

Atributos: `getAttribute`, `setAttribute`, `hasAttribute` e `removeAttribute`. `link.href` devolve a URL completa; `getAttribute("href")` devolve o que está escrito no HTML.

```
console.log(link.getAttribute("href"));     // "pagina.html"
link.setAttribute("target", "_blank");      // abre em nova aba
```

4 Navegando pela árvore

```
// <ul id="lista"> <li>Ana</li> <li id="bia">Bia</li> <li>Léo</li> </ul>
const bia = document.getElementById("bia");

bia.parentElement;            // <ul id="lista">  (o pai)
bia.parentElement.children;   // os 3 <li>        (os filhos)
bia.previousElementSibling;   // <li>Ana</li>     (irmão anterior)
bia.nextElementSibling;       // <li>Léo</li>     (próximo irmão)
```

> **Nas pontas, null:** não existe irmão antes do primeiro nem depois do último.

5 Eventos: reagindo ao usuário

`elemento.addEventListener(evento, função)`: a função (um **callback**, como no forEach) roda **toda vez** que o evento acontece. Eventos comuns: `click`, `input`, `submit`, `keydown`, `mouseover`.

```
const botao = document.querySelector("#btn");

botao.addEventListener("click", (evento) => {
  console.log("Clicou!", evento.target);
});
```

> **Sem parênteses:** passe `minhaFuncao`, e não `minhaFuncao()`. Com `()` ela roda na hora, em vez de esperar o clique.

O fluxo do DOM, quatro passos que aparecem em quase todo código: **selecionar → acessar → modificar → reagir**.

```
const btn = document.querySelector("#btn");          // 1. selecionar
const titulo = document.querySelector("#titulo");

btn.addEventListener("click", () => {                // 4. reagir
  const atual = titulo.textContent;                  // 2. acessar
  titulo.textContent = atual + "!";                  // 3. modificar
  titulo.classList.toggle("ativo");
});
```

6 Exercícios

Os exercícios seguem o fluxo da aula: do 1 ao 6 você seleciona e modifica; do 7 ao 12 reage a eventos. No site, a correção clica e digita sozinha para testar o seu código. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 Ler e trocar o título — SELECIONAR

Mostre no console o texto atual do `#titulo` e depois troque para "Olá, DOM!".

Saída esperada: Console: Green School · Página: Olá, DOM!

02 Listar os itens — SELECIONAR

Com `querySelectorAll(".item")` e `forEach`, mostre o texto de cada item da lista.

Saída esperada: HTML / CSS / JavaScript

03 Mudar o estilo — MODIFICAR

Deixe o `#titulo` verde e com 40px usando `style`.

Saída esperada: Título verde e grande

04 Classes — MODIFICAR

Adicione a classe `destaque` no card e remova a classe `oculto` do aviso.

Saída esperada: Card amarelo e aviso visível

05 Atributos do link — MODIFICAR

Mostre o href atual com `getAttribute`, troque para o site do MDN e faça abrir em nova aba.

Saída esperada: Console: pagina.html

06 Navegando pelos irmãos — NAVEGAR

A partir do `#bia`, mostre o irmão anterior, o próximo e o total de filhos do pai.

Saída esperada: Ana / Léo / Total: 3

07 Primeiro clique — EVENTOS

Ao clicar no `#btn`, troque o texto do `#mensagem` para "Botão clicado!".

Saída esperada: Botão clicado! (só depois do clique)

08 Contador — EVENTOS

Cada clique no `#mais` soma 1 no `#numero`. Use `Number()` para não juntar textos.

Saída esperada: 3 cliques → 3

09 Lendo o input

EVENTOS

Ao clicar em Enviar, mostre `` `Olá, ${nome.value}!` `` no `#saudacao`.

Saída esperada: Olá, Ana!

10 Card ativo

EVENTOS

A cada clique no card, alterne a classe `ativo` com `classList.toggle`.

Saída esperada: Liga e desliga

11 Cor de fundo

EVENTOS

A cada clique, pinte o `document.body` com a próxima cor do array e volte ao início depois da última.

Saída esperada: Uma cor por clique, em ciclo

12 Lista de compras

DESAFIO

Ao clicar em Adicionar, crie um `li` com o texto do input (se não estiver vazio) e limpe o campo.

Saída esperada: Novo item na lista

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.